

Multi-Scenario Payer Call Agent Core

Paid trial task · suggested time box: 1-2 focused days · stack of your choice

Why this exercise

This is a practical task that mirrors the kind of work the role actually involves: designing a reusable AI-driven agent that has to behave correctly when the real world is messy. It is deliberately open-ended. There is no single hidden 'right answer' - we are evaluating how you make architectural and judgment calls, how you handle failure, and how honestly you describe what you built. We pay for this task because it is real work and your time is worth respecting.

You do not need any healthcare background. Everything you need is in the primer below.

Domain primer (read once, then forget the jargon)

Healthcare providers constantly have to phone insurance companies ("payers"). Three of the most common calls:

- **Benefits verification:** before a patient's visit, confirm their insurance is active and what it covers - copay amount, remaining deductible, any visit limit, and whether a treatment needs prior approval.
- **Denied-claim follow-up:** a submitted bill was rejected with a reason code; call to find out exactly what is needed to get it paid.
- **Prior-authorization follow-up:** a request for pre-approval of treatment is pending; call to check its status and any missing items.

All three calls share a shape: dial in, navigate an automated phone menu (an "IVR" - the "press 1 for eligibility, press 2 for claims" tree), wait on hold, reach a human representative, ask a scripted set of questions, deal with the rep saying something unexpected, and come away with a structured answer. Today this is a person on hold for 20-40 minutes per call.

Your task

Build the reusable software **core** that automates this call, serving **all three scenarios from a single core driven by configuration - not three separate implementations**. Adding a fourth scenario later should be an obvious, small, fork-free change.

You do not need real telephony, a real payer, or any sensitive data. You build (or stub) a **mock payer simulator** that conforms to the small interface below, so everything runs locally with synthetic data.

The mock payer (build it, or stub to this interface)

```

class MockPayer:
    def ivr_prompt() -> str
        # current phone-menu text, e.g.
        # "Press 1 for eligibility, 2 for claims, 3 for auth..."

    def send_dtmf(digit: str) -> None
        # navigate the menu by 'pressing' a key

    def ask(question: str) -> str
        # ask the representative a question;
        # returns a free-text, sometimes messy, answer

    def call_state() -> str
        # one of: connected | on_hold | dropped | unreachable

```

Drive the simulator from **fixture files you author**. Each fixture defines: the IVR menu tree, the representative's answers keyed by question intent, and knobs to inject realistic failure. At minimum your fixtures must be able to produce: a clean successful call, an off-script/unexpected rep answer, a mid-call drop, a self-contradictory rep, an unreachable payer, and a "the rep transfers you to a different department" path. A suggested fixture shape (use anything equivalent):

```

scenario: benefits_verification
ivr:
    root: "Press 1 eligibility, 2 claims, 3 auth"
    "1": "Hold for the next representative..."
rep_answers:
    coverage_active: "Yeah that policy's active."
    copay: "Copay is twenty dollars."
    visit_limit: "Hmm I'm not seeing a visit limit on this."
    prior_auth: "That service does need a prior auth."
inject:
    drop_after_question: 3 # call drops mid-call
    contradict: { field: copay, second_answer: "actually it's $40" }
    transfer_on: claims # rep punts to another dept

```

Deliverables

- **The agent core and the contract/abstraction** that lets one core serve all three scenarios.
- **A conversation/turn loop:** navigate the IVR, ask the scenario's questions, parse answers, and recover gracefully when the rep goes off-script.
- **Three declarative scenario configs** (benefits / denial / auth). Demonstrate that a fourth scenario is a one-file addition.
- **An extraction + reconciliation step:** turn the messy call transcript into a structured result with a **confidence score**.
- **Failure handling:** a dropped call, an unreachable payer, or contradictory answers must produce an explicit *BLOCKED* / *needs-human* result. The core must never invent a successful answer.
- **Tests** covering the failure and ambiguity paths, plus 3-5 "golden transcript" fixtures with their expected structured output.

- **A short DESIGN.md (1-2 pages):** your architecture decisions; which steps call an LLM and why (and which deliberately do not); what you deferred and how you would approach it with more time.

Constraints (these are graded, not decoration)

- **Deterministic-first.** Script the rule-driven parts (menu navigation, known-question parsing). Use an LLM only where genuine reasoning is required - for example, interpreting an unexpected rep response. In DESIGN.md, list exactly which steps call the LLM and why.
- **Cost and latency aware.** Phone calls are latency-sensitive and LLM calls cost money per call at scale. State which model class you would route each LLM call to and the trade-off you are making.
- **Sensitive-data boundary.** The patient's insurance member ID is sensitive. It must not be passed in the clear across the MockPayer . ask boundary (treat that boundary as an external vendor). Handle this without us reminding you again.
- **No fabricated success.** An ambiguous or failed call yields an explicit blocked / needs-human result - never a guessed "benefits active" or "claim will be paid".
- **Synthetic data only.** No real payer credentials, no real patient data, no paid third-party services.

What we evaluate

We share this so you know what we value. None of it is gameable for the things that actually matter.

What we look at	What a strong submission shows
Abstraction judgment	One core plus three configs, with an obvious fork-free path to a fourth scenario - not three near-duplicate implementations.
AI vs. deterministic	The LLM is reserved for genuine ambiguity; everything reliably scriptable is scripted. The boundary is explained.
Failure & ambiguity	Dropped / unreachable / contradictory calls produce a clear blocked result. Nothing is fabricated to look successful.
System modeling	The call is modeled as an explicit state machine with retries and timeouts, not an untraceable chain of sequential calls.
Safety instinct	The sensitive-ID boundary is handled, ideally raised by you in DESIGN.md without further prompting.
Test discipline	Tests exercise the unhappy paths and golden transcripts, not only the happy path.
Honesty	DESIGN.md states clearly what you deferred and why, rather than implying completeness.

Ground rules

- **Synthetic data only.** Use only the mock/sample data described here or data you fabricate yourself. Do not use any real patient data, real insurance credentials, or any third-party paid service.

- **Time box.** Spend 1-2 focused days. If you run short on time, cut scope deliberately and say so in DESIGN.md rather than padding. We are not looking for a production system. A smaller, correct, honest solution is rated higher than a large one that overclaims or hides gaps.
- **Tech is your choice.** Any language/framework you are fluent in (TypeScript or Python preferred, but not required). No specific libraries are mandated.
- **AI assistants are allowed** (this is an AI-systems role and we use them too). You must fully understand and be able to explain every line you submit. Briefly note in your writeup where AI tooling materially shaped the solution.
- **Questions are welcome.** If something is ambiguous, make a reasonable assumption, write it down in your notes, and proceed - or email us. Choosing a sensible default and documenting it is itself part of what we evaluate.

How to submit

- A git repository (preferred) or a zip archive containing: source code, tests, any fixtures/sample data you added, your written notes document, and a single command (script or documented steps) to install and run everything from a clean checkout.
- Tell us roughly how much time you actually spent.
- Reply to the email this task came from with the repo link or attachment.

After you submit, we will schedule a ~45-minute conversation where you walk us through your design and we discuss trade-offs together. It is a discussion, not an interrogation - the goal is to see how you reason about systems, not to catch you out.

FAQ

- **Do I need a real LLM API key?** No. You may stub the LLM call behind an interface and document the prompt/inputs you would send. If you do call a real model, keep it minimal and do not commit any key.
- **How polished must the mock payer be?** Functional, not fancy. It exists so your core has something realistic-but-messy to talk to.
- **Front-end?** Not required. This is a systems/agent task; a CLI or test harness is fine.